



RESUMEN MIDTERM SSOO

Resumen Clase Lunes 28/10/24: Profe Lesly Reyes

S.O → Kernell + Programas

Kernell → Programa que siempre esta ejecutando

- Provee las funcionalidades minimas

SO

1. Nos abstrae del hardware
2. Permite compartir Recursos
3. Nos ofrece servicios

SO → Compuesto por

- Sistema de archivos
- Proteccion/Seguridad
- Control I/O

→ Drivers

Tipos de Kernells

1. Monolítico:

→ Complejo/grande

→ Rapido

2. MicroKernell:

→ Pequeño/modular

→ Aislado a fallos

Acceso

→ Kernell Mode

→ User Mode

Syscalls

Llaman al S.O

Proceso

Codigo + Recursos

Memoria Archivos

Sockets

CPU atiende "Simultaneamente" mas de un Proceso: **Multitasking**

Proceso

→ Codigo

→ Datos

→ Heap

→ Stack

PCB

Estado, ID

Estados de un proceso → New / Running / Ready / Terminated

Context Switch

Espacio de Swap

Threads = Procesos mas livianos

Syscalls

- Fork: Creacion de nuevo proceso
- Exec: Carga el binario en memoria
 - Reemplaza el codigo de quien lo llamo

Señales

Sigterm / Sigkill / Sigint

Wait → Espero del termino del proceso hijo

Proceso **Huerfano**: Cuando un padre termine (exit) o muere y sus hijos quedan huérfanos ()

Proceso **Zombie**: cuando un proceso termina y su padre **no hace wait**

Scheduling

Long Term (cola Ready) / Medium term (Swapping) / Short Term (selecciona ejecuta)

- Preemptie (expropiativo) : Saca el procesos de ejecucion
- Batch : Se ejecuta por lotes
- Interactivo : Minimiza el tiempo de respuesta

1. FCFS
2. SJF
3. RR (round robin)

Sincronizacion

→ Solucion al problema de la sección critica

→ Exclusion Mutua

→ Tiempo Acotado (espera)

→ Progreso

¿Que es una instrucción atomica?

→ Solo ejecuta una al mismo tiempo

MutexLock:

- Bloquea seccion critica del codigo con **AQUIRE**
- Libera seccion critica con **REALEASE**
- Solo permite un Thread por Seccion Critica

Semaforos:

- Permite N procesos en Sección Crítica
- VL → Aumenta contador
- PL → Disminuye contador. Si el contador está en 0, se **bloquea**

Variables de condición:

- Bloquea sección Crítica con Condición Arbitraria
- wait(): espera a Thread Libere Ejecución
- Signal(): Intento de **Despertar un Thread**

MEMORIA

Problemas :

1. **Relocalización**
2. **Protección**

Estrategias :

1. Segmentación
2. Paginación → IMPORTANTE !!! → Pregunta de prueba
3. Segmentos Paginados
4. Paginación Multinivel
5. Paginación Invertida

TLB

Es una memoria **CACHE** que almacena **traducciones** de Páginas → Frames

TLB HIT: Traducción encontrada

TLB MISS: Traducción no encontrada

→ Requiere ir a la memoria a buscar la traducción (tabla de traducciones)

Algoritmo de Reemplazo de Páginas :

- FIFO
- RANDOM
- LRU → Aproximaciones: CLOCK/ AGING / DIRTY BIT

ARCHIVOS

Disco Duro v/s SSD:

- SSD + Rapida
- SSD No tiene partes moviles
- Etc.

Capas de Bloques:

Tipos:

- Bloque de datos
- Bloque indice
- Bloque de indireccion simple/doble...

Estrategias:

- **ASIGNACION CONTIGUA**
- **ASIGNACION ENLAZADA**
- **FAT**
- [Asignacion indexada](#)
- [Asignacion indexada multinivel](#)

Estas ultimas 2 son importantes → Entraran si o si en la prueba

Comentarios de la profe

El midterm va a estar ordenado (del inicio de la materia hasta el fin)

S.O → Forma de administrar los recursos del computador / Nos brinda servicios para realizar nuestras tareas

Kernell: Provee de los servicios minimos:

Minimo: Cosas para administrar Hardware, Guardar archivos en memoria (sin un programa seria super dificil)

Controlador para detectar teclado, parlantes

Que el computador reconozca un puerto

Permite compartir recursos → ENTRE LOS PROCESOS

Tenemos Un disco duro, y tenemos varios procesos corriendo, por tanto tenemos que distribuir entre todos los recursos

Sistemas de archivos → El S.O permote Compartir la memoria (Generalmente esta en el KERNELL)

Proteccion Seguridad → En el KERNELL

Control I/O → Depende de la arquitectura

Procesos Intensivo de Input Output : Constantemente hace un accion y requiere de un input (esperan todo el rato que venga algo) → No usan tanta CPU, ejecuto algo que el usuario pidio , luego me salgo de la cpu, y espero a que el usuario haga algo para volver a entrar

- New → ready → Corriendo → Ready → →

Procesos de Computo: CPU Burst: No requieren de Input Output, por ejemplo programas de alto computo, correr una base de datos, es intensivo de computo, corre todo el rato en la CPU y cuando termine sale (no espera que el usuario haga algo)

Pregunta de prueba: Definir ventajas y desventajas de los Microkernel y monoliticos → Aprenderse las

Problema de monolitico: **PIEDRA** Como esta diseñado para trabajar todo junto, es bloqueante (-) (si falla algo), pero las llamadas son mas rapidas (+)

Microkernel: **LEGO** : Es modificable, mas tolerancia a fallas, es mas pequeño y modularizado, pero es mas lento, no esta diseñado para que funcione todo junto

Pregunta de prueba: Multitasking y multiprogramacion: (V o F)

simultaneamente entre comillas

Un core puede al mismo instante de tiempo puede correr mas de un proceso a la vez → **NO**

Lo que hacemos es mentir, y que va muy rapido

Este proceso se llama **Multitasking**

Kernell en 2 modos:

Modo kernell → Tiene toda las facultad de hacer todas las cosas

Modo usuario → El sistema op me provee de proteccion → No puedo borrar lo que yo quiera, no tengo todos los permisos

El usuario puede ser un super user (sudo), pero aun asi no tiene tantas facultades

Solo tiene las minimas para que el sistema funcione, y no interfiera

EJ: Suponga que tenemos que instalar denuevo un controlador, suponga que su mac esta instalando una nueva actualizacion, a pesar de que el usuario pida actualizar, el que tiene la facultad de hacerlo es el kernell

El sudo de linux, no te da acceso al anillo mas de adentro, sigue siendo un usuario

Solo el S.O puede meterse a hacer paginacion por ejemplo, moririamos si tuvieramos que hacer cosas asi

PREGUNTA SI O SI: V o F

Proceso:

- Código
- Heap
- Datos
- Stack

PCB: Guarda estado, Guarda el id (donde esta el codigo, cuanto tiempo a corrido, en que parte esta el program counter)

Siempre se pregunta como

La pcb permite guardar el estado : V

La pcb permite guardar el id: V

La pcb permite guardar estado del disco: F

Estados: New/ Running/ Waiting/ Ready/ Terminated

Context Switch: Correr el programa desde donde estaba (no partir desde el inicio)

Spacio de Swap: Donde van los programas que no se han estado usando por tiempo (ej: Si deje abierto el correo y no se ha usado en tiempo)

Espacio en memoria secundaria (disco), donde van los procesos que salen de la CPU

PREGUNTA SI O SI

Threads → ¿Que Comparten y que no comparten?

Fork

Exec

Pregunta: Definicion

Que es un proceso zombie?

Que es un proceos Huerfano?

Señales a saberse:

Sigterm (terminar en buena onda) / Sigkill (matarlo) / Sigint (interrupcion)

Pregunta si o si de schedulling

Ej: Hay 3 schedulling corriendose (long medium y short term)

Que hace el long → Escoge que procesos poner en la cola ready

Short term: Selecciono el que se va a ejecutar

Short term: Hace swapping: Saca de la Cpu

Distintos tipos de schedulers

Colaborativos → Sal cuando quieras → Los proecsos siempre querran terminar todo → Por ellos nunca salen

Expropiativos → Sacan los procesos de ejecucion → (Con un timer) , Estos son mas usados

Batch: Por lotes, y corre por ejemplo por las noches, en CPU BURST : Alto computo

Interactivo: Interactivo con el usuario (I/O)

Schedullers: tenemos que sabernos las definiciones

Los ejemplos son solo como los de la ayudantia → Aprenderse los

Si o si pregunta de sincronización

Aprenderse las formas de resolver una sección crítica

Siempre pensamos en 3 condiciones para sol

Exclusion mutua: Solo 1 thread y no mas de uno (max uno)

Tiempo acotado: No se queden infinitamente (algunos nunca entraran y mueren de hambre) (tiempo infinito)

Progreso: Asegura que siempre hay uno corriendo (todos) (almenos uno)

Instruccion atomicas → Compare and swap → Sabemos que estas insstrucciones solo se ejecuta esa al mismo tiempo

Una forma de proteger seccion criticas es:

- Semaforo
- Lock (acquire y realease) → Por debajo usa instrucciones atomicas

Mutex: Bloquea seccion de codigo

Libera SC con realease

Solo permiten 1 thread

Semaforos_

Paginacion es lo mas importante

→ Frame Fisico

→ Pagina → Virtual

Siempre se pregunta que es un hit y un miss → CACHE

Algoritmos de reemplazo de paginas → No va a ir tan cargado

Puede que vaya mas cargado

Un Disco duro no tiene aguja

Un harddrive si tiene

Las agujas son mas lentas, los ssd son mas rápida

El que se echa a perder es el de aguja si se cae al suelo

Asignaciones aprenderse

Los lonks aprenderse el simbolico y el hard

FOTOS PIZARRA

